

MULUNGUSHI UNIVERSITY



School of Engineering and Technology
Department of Computer Science and IT
ICT461 – Web Standards and HTTP Fundamentals

Task 3 Report: Investigating Caching and Browser Boundaries

Part 1 ETag and Cache-Control

1. Introduction

This report presents the evidence and analysis collected for Task 3 part 1 of the ICT461 lab: investigating HTTP caching behavior. Two areas were investigated: (3.1) conditional caching of GET /api/courses using ETag and Cache-Control, and the difference between browser and cURL behavior.

For each experiment the predicted result, the actual result, the supporting screenshot, and an explanation are recorded, in line with the Checkpoint B requirement.

2. Task 3.1 — HTTP Caching

GET /api/courses was configured on the server to return an ETag header (a hash of the current course list) and Cache-Control: public, max-age=60. This allows a client to cache the response for 60 seconds and then revalidate it cheaply using a conditional request, instead of redownloading the full body every time.

2.1 Baseline request

An unconditional GET request was sent first with cURL to capture the current representation and its ETag.

Figure 1: GET /api/courses returns 200 OK with ETag "e6467ef7e68292d52f3192b69f421e17" and Cache-Control: public, max-age=60.

2.2 Server configuration and data change

The course list in server.js was then edited to add a fourth course (ICT441 – Ethical Hacking). Because the ETag is derived from the response body, changing the underlying data produces a new ETag on the next request.

Figure 2: server.js — CORS origin-check function above the in-memory courses array, showing the added ICT441 Ethical Hacking record used to change the resource state.

2.3 Conditional request after the data changed

A second request was sent with the header `If-None-Match` set to the ETag captured in Figure 1 ("`e6467ef7e68292d52f3192b69f421e17`"). Because the course list had already changed, the server correctly did not treat the cached copy as fresh: it returned 200 OK with the full updated body (four courses, including Ethical Hacking) and a new ETag ("`185dfeca85e4f9debf3a8d7b95febe8`"), rather than a 304.



```
PS C:\Users\kondv\Downloads\ICT461_Lab> curl.exe --% -i -H "If-None-Match: \"e6467ef7e68292d52f3192b69f421e17\"" http://localhost:3000/api/courses
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: public, max-age=60
ETag: "185dfeca85e4f9debf3a8d7b95febe8"
Content-Type: application/json; charset=utf-8
Content-Length: 197
Date: Tue, 22 Sep 2026 09:58:57 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"code":"ICT461","name":"Web Standards & HTTP Fundamentals"}, {"code":"ICT411","name":"Software Engineering"}, {"code":"ICT321","name":"Database Systems"}, {"code":"ICT441","name":"Ethical Hacking"}]
PS C:\Users\kondv\Downloads\ICT461_Lab>
```

Figure 3: Conditional GET with a stale `If-None-Match` value returns 200 OK, the updated course list, and a new ETag — proving the server revalidates against the current resource rather than blindly trusting the client's cached tag.

2.4 Freshness versus revalidation

- **Freshness:** while a response is within its max-age window (60 seconds here), the browser's HTTP cache can reuse the stored copy without contacting the server at all, no request is even sent.
- **Revalidation:** once the response is stale (or the client forces a check), the browser sends a conditional GET with `If-None-Match`. If the ETag still matches, the server returns 304 Not Modified with no body, saving bandwidth while confirming the cached copy is still valid. If the resource changed, as shown above, the server returns 200 with the new representation and a new ETag.

Cache-Control: no-store was applied to all registration responses (POST/GET/PUT/PATCH/DELETE on `/api/registrations`), since student registration data is dynamic and must never be served from a stale cache — every request for it must reach the server.

5. Conclusion

Task 3 demonstrated two related but distinct browser/server boundaries. Caching (ETag + Cache-Control) lets a client avoid re-fetching unchanged data while still guaranteeing it sees new data as soon as the resource changes, and no-store was used to ensure registration data is never served stale. cURL bypasses this enforcement entirely, which made it a useful tool for isolating server-side origin logic from browser-side preflight behavior. In both areas, the fixes were made entirely through server response headers, without disabling any browser protections.